# Bandersnatch VRF-AD Specification

Davide Galassi Seyed Hosseini

28 Mar 2026 - Draft 32

---


## *Abstract*

This specification defines three Verifiable Random Function with Additional Data (VRF-AD) schemes – IETF VRF, Thin VRF, and Pedersen VRF – built on a transcript-based Fiat-Shamir transform with support for multiple input/output pairs via delinearization. The IETF VRF is based on RFC-9381 [1]; the Thin VRF and Pedersen VRF follow the constructions introduced by BCHSV23 [2], with the Pedersen VRF serving as a building block for anonymized ring signatures based on the ring proof scheme derived from CSSV22 [3]. All schemes are instantiated over the Bandersnatch elliptic curve, constructed over the BLS12-381 scalar field as specified in MSZ21 [4].


## 1. Preliminaries

### 1.1. Groups and Fields

- $\mathbf{G}$: Bandersnatch curve cyclic group of prime order $r$, defined over the base field of prime order $q$.
- $\mathbf{F}$: Scalar field of prime order $r$ (i.e. $\mathbb{Z}_r$).
- $\mathbf{\Sigma}^k$: Octet strings with length $k \in \mathbb{N}$ ($*$ for arbitrary length).
- $\mathcal{O}$: Identity point of $\mathbf{G}$.

The EC group $\mathbf{G}$ is the prime subgroup of the Bandersnatch elliptic curve, in Twisted Edwards form, with finite field and curve parameters as specified in MSZ21 [4]. For this group, `fLen` = `qLen` = 32 and `cofactor` = 4.

All point arithmetic MUST be performed in $\mathbf{G}$. Points not in $\mathbf{G}$ MUST be rejected at all entry points; accepting a point on the full curve but outside the prime-order subgroup enables small-subgroup attacks that break the VRF relation.

## 1.2. Notation

- $x \in \mathbf{F}$: Secret key scalar.
- $Y \in \mathbf{G}$: Public key point defined as $x \cdot G$.
- $i \in \mathbf{\Sigma}^*$: VRF input data.
- $I \in \mathbf{G}$: VRF input point.
- $O \in \mathbf{G}$: VRF output point.
- $o \in \mathbf{\Sigma}^k$: VRF output hash.
- $T$: Transcript state.

The *group generator* $G \in \mathbf{G}$ is defined as:

$$x = 18886178867200960497001835917649091219057080094937609519140440539760939937304$$
$$y = 19188667384257783945677642223292697773471335439753913231509108946878080696678$$

## 1.3. VRF-AD

Regardless of the specific scheme, a *Verifiable Random Function with Additional Data (VRF-AD)* can be concisely represented by three primary functions:

**Abstract interface**:

- $\texttt{prove}(x \in \mathbf{F}, \overline{io} \in (\mathbf{G} \times \mathbf{G})^n, ad \in \mathbf{\Sigma}^*) \to \Pi$
- $\texttt{verify}(Y \in \mathbf{G}, \overline{io} \in (\mathbf{G} \times \mathbf{G})^n, ad \in \mathbf{\Sigma}^*, \Pi) \to (\top \mid \bot)$
- $\texttt{output}(O \in \mathbf{G}) \to o \in \mathbf{\Sigma}^N$

For Pedersen VRF (section 4), the public key $Y$ is not an explicit input to `verify`; the proof $\Pi$ carries a blinded commitment $\bar{Y}$ instead.

The additional data $ad$ is an arbitrary-length octet-string signed together with the VRF output. It does not influence the produced VRF output. The length of $ad$ MUST NOT exceed $2^{32} - 1$ bytes, as the length is encoded via `enc_32` in the VRF transcript (section 1.6.5).

## 1.4. Constants

- `suite_id` = `[0x01, 0x01, 0x01, 0x01]` — a 4-byte fixed-width identifier encoding the protocol profile:

| Name | Byte | Value | Meaning |
|---|---|---|---|
| version | 0 | 0x01 | Suite version (v1) |
| curve | 1 | 0x01 | Bandersnatch |
| hash | 2 | 0x01 | SHA-512 |
| h2c | 3 | 0x01 | Elligator 2, random oracle |

The version byte bundles several tightly-coupled choices: transcript construction (HashTranscript), nonce algorithm (RFC-8032), challenge derivation (transcript squeeze), point encoding (compressed little-endian), and security level (128-bit). Bump this byte when any of these changes.

- `challenge_len` = 16 bytes (128-bit security).
- `expanded_scalar_len` = $\lceil(\lceil\log_2(r)\rceil + 128)/8\rceil = 48$ bytes.

Domain separation tags used throughout the protocol:

| Tag | Value | Usage |
|---|---|---|
| IetfVrf | 0x00 | IETF VRF scheme identifier |
| ThinVrf | 0x01 | Thin VRF scheme identifier |
| PedersenVrf | 0x02 | Pedersen VRF scheme identifier |
| NonceExpand | 0x10 | Nonce secret expansion |
| Nonce | 0x11 | Nonce derivation |
| PedersenBlinding | 0x12 | Pedersen blinding factor |
| PointToHash | 0x20 | VRF output hashing |
| Delinearize | 0x30 | Delinearization scalars |
| Challenge | 0x40 | Challenge derivation |
| ThinBatch | 0x50 | Thin VRF batch verification |
| PedersenBatch | 0x51 | Pedersen VRF batch verification |

## 1.5. Codec

- `enc_scalar`($s \in \mathbf{F}$): Encodes a scalar into 32 octets in little-endian representation.
- `dec_scalar`($buf \in \mathbf{\Sigma}^{32}$): Interpret octet string $buf$ as a little-endian integer. MUST output "INVALID" if the resulting value is $\geq r$.
- `enc_point`($P \in \mathbf{G}$): Encodes a point in compressed form. The $y$ coordinate is serialized in little-endian and the most significant bit of the last octet encodes the sign of the $x$-coordinate. This gives `ptLen` = `fLen` = 32.
- `dec_point`($buf \in \mathbf{\Sigma}^{32}$): Interpret octet string $buf$ as a compressed point. Mask the sign bit from the last octet and interpret the result as a little-endian integer. MUST output "INVALID" if the resulting value is $\geq q$. Otherwise, decompress the point and MUST output "INVALID" if it does not decode to a point on the prime subgroup $\mathbf{G}$.
- `dec_scalar_mod`($buf \in \mathbf{\Sigma}^*$): Interpret octet string $buf$ as a little-endian integer and reduce modulo the prime field order $r$.
- `enc_32`($n \in \mathbb{N}_{2^{32}}$): Encode integer $n$ as a 4-byte little-endian octet string.

Aggregate types (e.g. proofs) MUST be encoded as the concatenation of their individual fields in the order given by the structure definition, without any separator.

## 1.6. Procedures

### 1.6.1. Transcript

The transcript provides a Fiat-Shamir transform with an absorb/squeeze interface. Data is absorbed into an internal hash state; output bytes are squeezed from it. After the first squeeze, `absorb` MUST NOT be called.

**Abstract interface**:

- `new_transcript`() $\rightarrow T$: Create a fresh transcript instance and absorb `suite_id`.
- `absorb`($data \in \mathbf{\Sigma}^*$): Feed bytes into the hash state. MUST NOT be called after squeeze.
- `squeeze`($n \in \mathbb{N}$) $\rightarrow \mathbf{\Sigma}^n$: Produce $n$ output bytes.
- `fork`() $\rightarrow T$: Clone the transcript state.

A concrete instantiation using SHA-512 is given in Appendix A.1.

### 1.6.2. VRF Input

The VRF input point $I \in \mathbf{G}$ is derived from the input octet-string using a `hash_to_curve` function that maps arbitrary-length octet-strings to points in $\mathbf{G}$.

$$I \leftarrow \texttt{hash_to_curve}(i)$$

The function MUST behave as a random oracle: its output must be indistinguishable from a uniformly random point in $\mathbf{G}$, and the discrete logarithm of the output with respect to any known base must be unknown.

Verifiers MUST independently compute each $I_i$ from the corresponding input octet-string using the procedure above. Accepting prover-supplied input points without recomputation breaks the VRF security guarantees, and in the case of Thin VRF (section 3), enables trivial forgery.

A concrete instantiation using Elligator 2 is given in Appendix A.2.

### 1.6.3. VRF Output

The VRF output point is generated from the VRF input point and secret key scalar:

$$O \leftarrow x \cdot I$$

The VRF output hash is a fixed-length octet string derived from the output point using a transcript-based point-to-hash procedure. The procedure is deliberately independent of the proof scheme: for a given key and input, the output point $O = x \cdot I$ is unique regardless of whether IETF VRF, Thin VRF, or Pedersen VRF is used to prove correctness. The scheme determines how the proof is constructed, not the VRF output itself. This separation allows applications to obtain consistent output hashes across schemes for the same underlying evaluation.

**Input**:

- $O \in \mathbf{G}$: VRF output point.
- $N \in \mathbb{N}$: Desired output length in bytes. MUST be fixed per application context; $N$ is not absorbed into the transcript, so $\texttt{squeeze}(N_1)$ is a prefix of $\texttt{squeeze}(N_2)$ for $N_1 < N_2$.

**Output**:

- $o \in \mathbf{\Sigma}^N$: VRF output hash.

**Steps**:

1. $T \leftarrow \texttt{new_transcript}()$
2. $T.\texttt{absorb}(\texttt{PointToHash} \parallel \texttt{enc_point}(O))$
3. $o \leftarrow T.\texttt{squeeze}(N)$

**Transcript**:

- $T = \texttt{suite_id} \parallel \texttt{PointToHash} \parallel \texttt{enc_point}(O)$

### 1.6.4. Delinearization

Merges input/output pairs into a single pair using delinearization scalars derived from the transcript. For $n = 0$, returns the identity pair. For $n = 1$, the pair is returned unchanged ($z_0 = 1$). For $n \geq 2$, random scalars prevent an attacker from mixing components across pairs.

**Input**:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: Sequence of input/output pairs.
- $T$: Transcript state.

**Output**:

- $(I_m, O_m) \in \mathbf{G} \times \mathbf{G}$: Merged input/output pair.

**Steps**:

1. If $n = 0$: return $(\mathcal{O}, \mathcal{O})$
2. $T$.`absorb`(`Delinearize`)
3. $z_0 \leftarrow 1$
4. For $i = 1, \ldots, n-1$: $z_i \leftarrow$ `dec_scalar_mod`($T$.`squeeze`(`challenge_len`))
5. $I_m \leftarrow \sum_{i=0}^{n-1} z_i \cdot I_i,\ O_m \leftarrow \sum_{i=0}^{n-1} z_i \cdot O_i$
6. Return $(I_m, O_m)$

**Transcript** (where $T_{in}$ is the caller-supplied state):

- $T = T_{in} \parallel$ `Delinearize`

### 1.6.5. VRF Transcript

Shared transcript construction used by all VRF-AD schemes. Absorbs input/output pairs, merges them via delinearization (section 1.6.4), and absorbs additional data.

**Input**:

- *scheme*: Scheme identifier tag.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: Sequence of input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

**Output**:

- $T$: Transcript state.
- $(I_m, O_m) \in \mathbf{G} \times \mathbf{G}$: Merged input/output pair.

**Steps**:

1. $T \leftarrow$ `new_transcript`()
2. $T$.`absorb`(*scheme*)
3. $T$.`absorb`(`enc_32`($n$))
4. For each $(I_i, O_i)$ in $\overline{io}$: $T$.`absorb`(`enc_point`($I_i$) $\parallel$ `enc_point`($O_i$))
5. $T$.`absorb`(`enc_32`(`len`($ad$)) $\parallel$ $ad$)
6. $(I_m, O_m) \leftarrow$ `delinearize`($\overline{io}, T$.`fork`())
7. Return $(T, (I_m, O_m))$

**Transcript**:

$$T = \texttt{suite_id} \parallel scheme \parallel \texttt{enc_32}(n) \parallel \texttt{enc_point}(I_0) \parallel \texttt{enc_point}(O_0) \parallel \cdots \parallel \texttt{enc_point}(I_{n-1}) \parallel \texttt{enc_point}(O_{n-1}) \parallel \texttt{enc_32}(\texttt{len}(ad)) \parallel ad$$

### 1.6.6. Nonce

Deterministic nonce generation inspired by [RFC-8032] section 5.1.6. The transcript carries shared state from `vrf_transcript`, binding the nonce to the I/O pairs and additional data.

**Input**:

- $d \in \mathbf{F}$: Secret scalar.
- $T$: Transcript state.

**Output**:

- $k \in \mathbf{F}$: Nonce scalar.

**Steps**:

1. $T' \leftarrow T.\texttt{fork}()$
2. $T'.\texttt{absorb}(\texttt{NonceExpand} \parallel \texttt{enc_scalar}(d))$
3. $h \leftarrow T'.\texttt{squeeze}(64)$
4. $T.\texttt{absorb}(\texttt{Nonce} \parallel h)$
5. $k \leftarrow \texttt{dec_scalar_mod}(T.\texttt{squeeze}(\text{expanded_scalar_len}))$
6. If $k = 0$: abort (implementation error; probability $\approx 2^{-253}$).

**Transcript** (where $T_{in}$ is the caller-supplied state):

- $T' = T_{in} \parallel \texttt{NonceExpand} \parallel \texttt{enc_scalar}(d)$
- $T = T_{in} \parallel \texttt{Nonce} \parallel h$

### 1.6.7. Challenge

Derives a challenge scalar by absorbing curve points into the transcript and squeezing.

**Input**:

- $\bar{P} \in \mathbf{G}^m$: Sequence of $m$ points.
- $T$: Transcript state.

**Output**:

- $c \in \mathbf{F}$: Challenge scalar.

**Steps**:

1. $T.\texttt{absorb}(\texttt{Challenge})$
2. For each $P_i$ in $\bar{P}$: $T.\texttt{absorb}(\texttt{enc_point}(P_i))$
3. $c \leftarrow \texttt{dec_scalar_mod}(T.\texttt{squeeze}(\texttt{challenge_len}))$

**Transcript** (where $T_{in}$ is the caller-supplied state):

- $T = T_{in} \parallel \texttt{Challenge} \parallel \texttt{enc_point}(P_0) \parallel \cdots \parallel \texttt{enc_point}(P_{m-1})$

# 2. IETF VRF

Based on IETF RFC-9381 but adapted with a transcript-based Fiat-Shamir transform, support for additional data ($ad$), and multiple I/O pairs via de-linearization. These changes make this scheme incompatible with standard RFC-9381 implementations and test vectors.

## 2.1. Prove

**Input**:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

**Output**:

- $\pi = (c, s) \in (\mathbf{F}, \mathbf{F})$: Schnorr-like proof.

**Steps**:

1. $Y \leftarrow x \cdot G$
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{IetfVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $k \leftarrow \texttt{nonce}(x, T.\texttt{fork}())$
4. $U \leftarrow k \cdot G, \quad V \leftarrow k \cdot I_m$
5. $c \leftarrow \texttt{challenge}([U, V], T)$
6. $s \leftarrow k + c \cdot x$
7. $\pi \leftarrow (c, s)$

## 2.2. Verify

**Input**:

- $Y \in \mathbf{G}$: Public key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (c, s) \in (\mathbf{F}, \mathbf{F})$: Schnorr-like proof.

**Output**:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

**Steps**:

1. Validate $Y$ and all $I_i, O_i \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\bot$ if any is invalid or the identity.
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{IetfVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $U \leftarrow s \cdot G - c \cdot Y$
4. $V \leftarrow s \cdot I_m - c \cdot O_m$
5. $c' \leftarrow \texttt{challenge}([U, V], T)$
6. $\theta \leftarrow \top$ if $c = c'$ else $\bot$

# 3. Thin VRF

Thin VRF is derived from the PedVRF construction in section 4 of BCHSV23 [2] by removing the blinding mechanism entirely (see remark on page 13 of the paper). Without blinding, Pedersen VRF reduces to two independent DLEQ checks on $(G, Y)$ and $(I_m, O_m)$ with the same secret $x$. Thin VRF merges these into a single DLEQ relation by prepending $(G, Y)$ to the I/O pairs and applying delinearization, then proves it with a Schnorr-like proof $(R, s)$. Storing the nonce commitment $R$ (rather than the challenge $c$) enables batch verification.

**Security**: VRF input points MUST be constructed via hash-to-curve. If a prover knows $d$ such that $I = d \cdot G$, they can forge arbitrary outputs for that input, because the delinearization merges the Schnorr and VRF pairs into a single check that collapses when all points are multiples of $G$.

## 3.1. Prove

**Input**:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

**Output**:

- $\pi = (R, s) \in (\mathbf{G}, \mathbf{F})$: Thin VRF proof.

**Steps**:

1. $Y \leftarrow x \cdot G$
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{ThinVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $k \leftarrow \texttt{nonce}(x, T.\texttt{fork}())$

4. $R \leftarrow k \cdot I_m$
5. $c \leftarrow \texttt{challenge}([R], T)$
6. $s \leftarrow k + c \cdot x$
7. $\pi \leftarrow (R, s)$

## 3.2. Verify

**Input**:

- $Y \in \mathbf{G}$: Public key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (R, s) \in (\mathbf{G}, \mathbf{F})$: Thin VRF proof.

**Output**:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

**Steps**:

1. Validate $Y$, $R$, and all $I_i, O_i \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\bot$ if any is invalid or the identity.
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{ThinVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $c \leftarrow \texttt{challenge}([R], T)$
4. $\theta \leftarrow \top$ if $s \cdot I_m = R + c \cdot O_m$ else $\bot$

## 3.3. Batch Verify

Multiple Thin VRF proofs can be verified together by combining the individual verification equations with random weights (Schwartz-Zippel lemma).

**Input**:

- For $j = 0, \ldots, N-1$: a tuple $(Y_j, \overline{io}_j, ad_j, \pi_j)$ where:
  - $Y_j \in \mathbf{G}$: Public key.
  - $\overline{io}_j \in (\mathbf{G} \times \mathbf{G})^{M_j}$: VRF input/output pairs.
  - $ad_j \in \mathbf{\Sigma}^*$: Additional data octet-string.
  - $\pi_j = (R_j, s_j) \in (\mathbf{G}, \mathbf{F})$: Thin VRF proof.

**Output**:

- $\theta \in \{\top, \bot\}$: $\top$ if all proofs verify, $\bot$ otherwise.

**Steps**:

1. For each proof $j$:
   (a) Validate $Y_j$, $R_j$, and all $I_{j,i}, O_{j,i} \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\perp$ if any is invalid or the identity.
   (b) $(T_j, (I_{m,j}, O_{m,j})) \leftarrow \texttt{vrf_transcript}(\texttt{ThinVrf}, [(G, Y_j)] \parallel \overline{io}_j, ad_j)$
   (c) $c_j \leftarrow \texttt{challenge}([R_j], T_j)$
2. Derive random weights:
   (a) $T_w \leftarrow \texttt{new_transcript}()$
   (b) $T_w.\texttt{absorb}(\texttt{ThinBatch})$
   (c) For each $j$: $T_w.\texttt{absorb}(\texttt{enc_scalar}(c_j) \parallel \texttt{enc_scalar}(s_j))$
3. Check the combined equation:

$$\sum_{j=0}^{N-1} w_j \cdot (s_j \cdot I_{m,j} - R_j - c_j \cdot O_{m,j}) = \mathcal{O}$$

   where $w_j \leftarrow \texttt{dec_scalar_mod}(T_w.\texttt{squeeze}(\texttt{challenge_len}))$.

**Transcript**:

$$T_w = \texttt{suite_id} \parallel \texttt{ThinBatch} \parallel \texttt{enc_scalar}(c_0) \parallel \texttt{enc_scalar}(s_0) \parallel \cdots \parallel \texttt{enc_scalar}(c_{N-1}) \parallel \texttt{enc_scalar}(s_{N-1})$$

# 4. Pedersen VRF

Pedersen VRF resembles IETF EC-VRF but replaces the public key with a Pedersen commitment to the secret key, which makes this VRF useful in anonymized ring proofs.

The scheme proves that the output has been generated with a secret key associated with a blinded public key (instead of the public key). The blinded public key is a cryptographic commitment to the public key, and it can be unblinded to prove that the output of the VRF corresponds to the public key of the signer.

This specification mostly follows the design proposed by BCHSV23 [2] in section 4 with some details about blinding base point value and challenge generation procedure.

The *blinding base* $B \in \mathbf{G}$ is defined as:

$x = 52264259925712207693658434871020643071012729807919931342737807369975449493 82$
$y = 46544868206883149332782258938702216106598247683423727002885664111567608220426$

A point with unknown discrete logarithm derived using the `hash_to_curve` function as described in Appendix A.2 with input the string: `"pedersen-blinding"`.

## 4.1. Prove

**Input**:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

**Output**:

- $\pi = (\bar{Y}, R, O_k, s, s_b) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $b \in \mathbf{F}$: Blinding factor.

**Steps**:

1. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{PedersenVrf}, \overline{io}, ad)$
2. $b \leftarrow \texttt{blinding}(x, T.\texttt{fork}())$ (see Appendix A.4)
3. $\bar{Y} \leftarrow x \cdot G + b \cdot B$
4. $T.\texttt{absorb}(\texttt{enc_point}(\bar{Y}))$
5. $k \leftarrow \texttt{nonce}(x, T.\texttt{fork}()), \quad k_b \leftarrow \texttt{nonce}(b, T.\texttt{fork}())$
6. $R \leftarrow k \cdot G + k_b \cdot B$
7. $O_k \leftarrow k \cdot I_m$
8. $c \leftarrow \texttt{challenge}([R, O_k], T)$
9. $s \leftarrow k + c \cdot x, \quad s_b \leftarrow k_b + c \cdot b$
10. $\pi \leftarrow (\bar{Y}, R, O_k, s, s_b)$

## 4.2. Verify

**Input**:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (\bar{Y}, R, O_k, s, s_b) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.

**Output**:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

**Steps**:

1. Validate $\bar{Y}$, $R$, $O_k$, and all $I_i, O_i \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\bot$ if any is invalid or the identity.
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{PedersenVrf}, \overline{io}, ad)$

3. $T.\texttt{absorb}(\texttt{enc_point}(\bar{Y}))$
4. $c \leftarrow \texttt{challenge}([R, O_k], T)$
5. $\theta_0 \leftarrow \top$ if $O_k + c \cdot O_m = s \cdot I_m$ else $\perp$
6. $\theta_1 \leftarrow \top$ if $R + c \cdot \bar{Y} = s \cdot G + s_b \cdot B$ else $\perp$
7. $\theta = \theta_0 \wedge \theta_1$

Note: no public key appears in the verify inputs – verification uses the committed key $\bar{Y}$ from the proof.

## 4.3. Unblinding

To link a Pedersen VRF proof to a specific public key, the prover reveals the blinding factor $b$ and the verifier checks:

$$\bar{Y} = Y + b \cdot B$$

where $Y \in \mathbf{G} \setminus \{\mathcal{O}\}$ is the claimed public key. The verifier MUST validate $Y$ before accepting the association.

## 4.4. Batch Verify

Multiple Pedersen VRF proofs can be verified together by combining the individual verification equations with random weights (Schwartz-Zippel lemma). Each proof contributes two equations (VRF correctness and Pedersen commitment correctness), each weighted by an independent random scalar.

**Input**:

- For $j = 0, \ldots, N-1$: a tuple $(\overline{io}_j, ad_j, \pi_j)$ where:
  - $\overline{io}_j \in (\mathbf{G} \times \mathbf{G})^{M_j}$: VRF input/output pairs.
  - $ad_j \in \mathbf{\Sigma}^*$: Additional data octet-string.
  - $\pi_j = (\bar{Y}_j, R_j, O_{k,j}, s_j, s_{b,j}) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.

**Output**:

- $\theta \in \{\top, \perp\}$: $\top$ if all proofs verify, $\perp$ otherwise.

**Steps**:

1. For each proof $j$:
   (a) Validate $\bar{Y}_j$, $R_j$, $O_{k,j}$, and all $I_{j,i}, O_{j,i} \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\perp$ if any is invalid or the identity.
   (b) $(T_j, (I_{m,j}, O_{m,j})) \leftarrow \texttt{vrf_transcript}(\texttt{PedersenVrf}, \overline{io}_j, ad_j)$
   (c) $T_j.\texttt{absorb}(\texttt{enc_point}(\bar{Y}_j))$

(d) $c_j \leftarrow \texttt{challenge}([R_j, O_{k,j}], T_j)$

2. Derive random weights:

   (a) $T_w \leftarrow \texttt{new_transcript}()$

   (b) $T_w.\texttt{absorb}(\texttt{PedersenBatch})$

   (c) For each $j$: $T_w.\texttt{absorb}(\texttt{enc_scalar}(c_j) \parallel \texttt{enc_scalar}(s_j) \parallel \texttt{enc_scalar}(s_{b,j}))$

3. Check the combined equations:

$$\sum_{j=0}^{N-1} t_j \cdot (O_{k,j} + c_j \cdot O_{m,j} - s_j \cdot I_{m,j}) + u_j \cdot (R_j + c_j \cdot \bar{Y}_j - s_j \cdot G - s_{b,j} \cdot B) = \mathcal{O}$$

   where:

   - $t_j \leftarrow \texttt{dec_scalar_mod}(T_w.\texttt{squeeze}(\texttt{challenge_len}))$
   - $u_j \leftarrow \texttt{dec_scalar_mod}(T_w.\texttt{squeeze}(\texttt{challenge_len}))$

**Transcript**:

$$\begin{aligned} T_w = \; & \texttt{suite_id} \parallel \texttt{PedersenBatch} \\ & \parallel \texttt{enc_scalar}(c_0) \parallel \texttt{enc_scalar}(s_0) \parallel \texttt{enc_scalar}(s_{b,0}) \\ & \parallel \cdots \\ & \parallel \texttt{enc_scalar}(c_{N-1}) \parallel \texttt{enc_scalar}(s_{N-1}) \parallel \texttt{enc_scalar}(s_{b,N-1}) \end{aligned}$$

# 5. Ring VRF

Anonymized ring VRF based on Pedersen VRF (section 4) and Ring Proof as proposed in BCHSV23 [2].

The ring proof can be seen as a special case of the Committee Key Scheme (CKS) introduced by CSSV22 [3], reduced to a single signer. In CKS, a prover commits to a set of public keys using a KZG polynomial commitment and produces a SNARK showing that a subset of keys – identified by a bitvector – belongs to the committed set. The ring proof is the degenerate case where the bitvector has exactly one bit set: it proves that a single (blinded) key is a member of the committed ring, without revealing which one.

The concrete specification of the ring proof scheme is given in VG24 [5]. The following configuration specializes it for this scheme:

- **Groups and Fields**:
  - $\mathbb{G}_{\mathbb{K}}$: BLS12-381 prime order subgroup.
  - $\mathbb{F}$: BLS12-381 scalar field.
  - $J$: Bandersnatch curve defined over $\mathbb{F}$.
- **Polynomial Commitment Scheme**

  - KZG with SRS derived from Zcash powers of tau ceremony.

- **Fiat-Shamir Transform**
  - `ark-transcript`.
  - Begin with empty transcript and "ring-proof" label.
  - Push $R$ to the transcript after instancing.

- Accumulator base point $S \in \mathbf{G}$ is defined as:
  $x = 42303668360647658687880456753606405401141031996216729331450763906967498848487$
  $y = 41898972259388202032055565840730004413653698329702630697317353721966090663285$

A point with unknown discrete logarithm derived using the `hash_to_curve` function as described in Appendix A.2 with input the string: `"ring-accumulator"`.

- Padding point $\square \in \mathbf{G}$ is defined as:
  $x = 29586100106858075217954567072572265001347911471605742544678436487322334776392$
  $y = 21753411410084671346581650250322348778806357231808407562422401169820213423498$

A point with unknown discrete logarithm derived using the `hash_to_curve` function as described in Appendix A.2 with input the string: `"ring-padding"`.

- Polynomials domain ($\langle\omega\rangle = \mathbb{D}$) generator:
  $\omega = 49307615728544765012166121802278658070711169839041683575071795236746050763237$

- $|\mathbb{D}| = 2048$

## 5.1. Prove

**Input**:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $P$: Ring prover (encapsulates ring keys and prover index).

**Output**:

- $\pi_p \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $\pi_r \in ((G_1)^4, (\mathbf{F})^7, G_1, \mathbf{F}, G_1, G_1)$: Ring proof.

**Steps**:

1. $(\pi_p, b) \leftarrow \mathit{Pedersen.prove}(x, \overline{io}, ad)$
2. $\pi_r \leftarrow \mathit{Ring.prove}(P, b)$

The blinding factor $b$ is derived internally by Pedersen prove (section 4.1, step 2) and forwarded to the ring prover. *Ring.prove* and *Ring.verify* are defined in VG24 [5].

### 5.2. Verify

**Input**:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi_p \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $\pi_r \in ((G_1)^4, (\mathbf{F})^7, G_1, \mathbf{F}, G_1, G_1)$: Ring proof.
- $V \in (G_1)^3$: Ring verifier (pre-processed commitment).

**Output**:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

**Steps**:

1. $\theta_0 \leftarrow Pedersen.verify(\overline{io}, ad, \pi_p)$
2. $(\bar{Y}, R, O_k, s, s_b) \leftarrow \pi_p$
3. $\theta_1 \leftarrow Ring.verify(V, \pi_r, \bar{Y})$
4. $\theta \leftarrow \theta_0 \wedge \theta_1$

# Appendix A. Concrete Instantiations

The following are concrete instantiations of the abstract interfaces defined in the main specification. They are provided to enable interoperable implementations and reproducible test vectors. Alternative constructions that satisfy the same security requirements are equally valid.

## A.1. Transcript Construction

Instantiation of the transcript interface (section 1.6.1) using SHA-512.

**Initialization**: `new_transcript()` creates a fresh SHA-512 state and feeds `suite_id` into it.

**Absorb**: feeds raw bytes directly into the SHA-512 state. Consecutive absorb calls are equivalent to a single absorb of the concatenated data. This is safe because all protocol fields use fixed-width encoding (`enc_point`: 32 bytes, `enc_scalar`: 32 bytes, `enc_32`: 4 bytes, domain tags: 1 byte) or explicit length prefixing ($ad$ via $\texttt{enc_32}(\texttt{len}(ad)) \parallel ad$), so the byte stream is unambiguous given the inputs agreed upon by both parties.

**Squeeze** (counter-mode XOF): on the first squeeze call, finalize the SHA-512 state to obtain a 64-byte *seed*. Then produce output blocks:

$$block_i = \text{SHA-512}(seed \parallel \texttt{enc_32}(i)) \quad \text{for } i = 0, 1, 2, \ldots$$

Each block yields 64 bytes. Output is read sequentially across blocks; partial block state is preserved between squeeze calls.

**Fork**: duplicates the full internal state (including any partial block position if squeezing has begun).

## A.2. Hash to Curve

Instantiation of the `hash_to_curve` function (section 1.6.2) using the method defined in section 3 of RFC-9380 [6], with the *Elligator 2* map to curve (section 6.8.2) and `expand_message_xmd` with SHA-512 (section 5.3.1).

This is the random oracle (`_RO_`) construction: the input is hashed to two independent field elements, each is mapped to a curve point via Elligator 2, and the results are added.

$$I \leftarrow \texttt{hash_to_curve_ell2}(DST, i)$$

The domain separation tag is:

$$DST = \text{"ECVRF_"} \parallel \texttt{h2c_suite_id} \parallel \texttt{suite_id}$$

where `h2c_suite_id` = `"Bandersnatch_XMD:SHA-512_ELL2_RO_"` is the RFC-9380 suite identifier string determined by the curve, hash, and h2c bytes of `suite_id`.

## A.3. Secret Key Generation

Derives a secret scalar from a 32-byte seed.

**Input**:

- $seed \in \mathbf{\Sigma}^{32}$: seed octet-string.

**Output**:

- $x \in \mathbf{F}$: secret key scalar.

**Steps**:

1. $i \leftarrow 0$
2. $T \leftarrow \texttt{new_transcript}()$
3. $T.\texttt{absorb}(seed)$
4. If $i > 0$: $T.\texttt{absorb}(i)$ where $i$ is encoded as a single octet
5. $d \leftarrow \texttt{dec_scalar_mod}(seed)$
6. $x \leftarrow \texttt{nonce}(d, T)$

7. If $x = 0$: increment $i$ and go to step 2
8. Return $x$

The seed is absorbed into the transcript and also passed as a scalar to the `nonce` procedure (section 1.6.6), ensuring seed entropy flows through both the transcript state and the secret scalar input paths.

### A.4. Blinding Factor Generation

Generates the Pedersen VRF blinding factor deterministically from the secret key and the VRF transcript state, using the nonce function (section 1.6.6) with a distinct domain separator.

**Linkability warning**: because $b$ is derived deterministically from $(x, T)$, two Pedersen VRF proofs with the same secret key, I/O pairs, and additional data will produce the same blinding factor $b$ and therefore the same blinded public key $\bar{Y} = x \cdot G + b \cdot B$. An observer can detect that both proofs originate from the same signer by comparing $\bar{Y}$ values. Applications that require unlinkability across repeated proofs on the same inputs should generate $b$ as a fresh uniformly random scalar rather than using this deterministic method.

**Input**:

- $x \in \mathbf{F}$: Secret scalar.
- $T$: Transcript state (from `vrf_transcript`).

**Output**:

- $b \in \mathbf{F}$: Blinding factor scalar.

**Steps**:

1. $T$.`absorb`(`PedersenBlinding`)
2. $b \leftarrow$ `nonce`$(x, T)$

## Appendix B. Behavior with Zero I/O Pairs

When $n = 0$ no VRF output can be derived, since there are no output points to hash. The proof-of-knowledge component, however, remains sound in all schemes: a valid proof still requires knowledge of the secret key $x$.

- **IETF VRF and Thin VRF**: Both schemes prepend the Schnorr pair $(G, Y)$ to the I/O list before delinearization (sections 2.1 and 3.1, step 2), so the internal pair count is at least 1 regardless of the user-supplied $n$. With zero VRF pairs, the scheme degenerates to a Schnorr signature on the additional data $ad$, proving knowledge of $x$ for public key $Y$.
- **Pedersen VRF**: No Schnorr pair is prepended, so with $n = 0$ the `delinearize` procedure (section 1.6.4) sets the merged pair to the identity:

$(I_m, O_m) = (\mathcal{O}, \mathcal{O})$. The VRF output check $O_k + c \cdot O_m = s \cdot I_m$ is vacuously satisfied ($\mathcal{O} = \mathcal{O}$), but the commitment check $R + c \cdot \bar{Y} = s \cdot G + s_b \cdot B$ still proves knowledge of the Pedersen commitment opening $(x, b)$.

# Appendix C. Test Vectors

The test vectors in this section were generated using `ark-vrf` libraries revision `c01eee3`.

## C.1. IETF VRF Test Vectors

Schema:

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
proof_c: Proof 'c' component,
proof_s: Proof 's' component,
```

### bandersnatch_sha-512_ell2_ietf - vector-1

```
02bc4e98404607ce56429007e6ec20a2c0cb2070fd77bb377d25c83afff4670c,
05727c7bdb1a03a985bac26b26f4840730336e9f1aedcdc829e0e95ae5e80936,
-,
-,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
45ee3e316c29dc0d5c2490ad14f4c1ebafbedf897d3663406e08acaef865a586,
b963d3ab9dd2e29a172aee407cb18def053e08d43d6b50bf04544c448a3edcd6,
6032c5002f9cb3ccda4f29086798bacf,
2fa2c286ff35bd29bc2c5f7f7fa5c4032766ca4e95874b14b1624e88b223ff08,
```

### bandersnatch_sha-512_ell2_ietf - vector-2

```
c5410a37c4dac5e7a0f02c2df9073a70b8c16d9c786c054b70f7502741c64f14,
dc9f6c647b3fe0b3248d6afb2d305d75b421b9e697e0eaff913e59dae68a76cc,
0a,
-,
a208afb3b276ef91530ce906abe1e64917b612b6e062a0bb93090c77d9c5ba95,
331bc541947a27c518a7101db25b20878bc075ccdbf15f22243d4b7cf87cd8bb,
24884e2a38ba6a7bdd6016be94cf09bc26306c0d649c67d4ed5e3ecd07ed690d,
8171a0d99191fb5fc2995421d3af66cc,
e625c8bd52dddb57199521c1f13176b1765e163a89cbe874db23ce341d9b8601,
```

**bandersnatch_sha-512_ell2_ietf - vector-3**

```
783510ef50915b52075650aa994085d36d513e259115fa408ac67f1398814605,
689ccd15496386db306024633493e60135a2673faad2b0b4140a4006537c71b8,
-,
0b8c,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
28c86b6880842180b972a94696703d6b489f8f03a3c280d9d3047017380e0baa,
1c8c9baf9d6b691be3db3302bd64c56880f9468bb4e98ffcb545a56876717fd3,
10948ba42b50dabb71aff44868e7ef4f,
67e5471d81aa5eaab68a89a93094b751f72f9834693cfc392aad75a7b71ac302,
```

**bandersnatch_sha-512_ell2_ietf - vector-4**

```
ac162d824faac7fb45ffd6f52acc819537139a5a3027e16d2736b84e3904301c,
566dd2374ae7621fb6910143f1f5fa91b7bfce2db27a731cf110a27309b94667,
73616d706c65,
-,
f534e99a16886cb60e3672a9bcd65b57ec8a76a3d3f005850e9b38ea17d81636,
b7d8e88cdb0dfe99bba76e5693d4d562c7fa8cd06d84502582aea9f8ab0b0d8a,
8f006754011b549653e3796fda62c5fd9d0d3c5913e1737fdb638e9bf53879d2,
6d2f2bfb41fe81f77b2ee671cbe2183f,
b5c4a230641cbbf0decb7d8d916b01e6d6db9cf0157104c2cf24a12829e7150b,
```

**bandersnatch_sha-512_ell2_ietf - vector-5**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
-,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
a82255b6b29d70f2d024d90246212d6a,
83c2b1baf0e8cd05386bc98a81c8dec06ad5e383940c593364b0b6de2fa9fd0d,
```

**bandersnatch_sha-512_ell2_ietf - vector-6**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
1f00564908115c4162eead4136f371bb,
b5d08cd00512786bccc3a87179b13382bfb44b371527fc42007fdbe44fcc1f01,
```

**bandersnatch_sha-512_ell2_ietf - vector-7**

```
76f8b81d866b4c8b89d1f7d40954c406ad3b6c33e2bb8ece9102d7a4f8483502,
0384ac15569e3147105862c293863fa47aed4718b5fd791cc689fbfa7e8ce75c,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
a2332c9646371ec14e3ba6ac3ae8ea93a65159dec7dcd2e5c022e14bdf7395b3,
26ea4ea18ba21e48b1c0a658fc1a4600f21c6957f6cc85e1209e69b77dc1eab6,
e6ff791f65e3f3b2cd502ebf75726c1a,
77a4db336448448b3e611e37c01290dd08132d0263c67249a25f34d2e461f31c,
```

## C.2. Thin VRF Test Vectors

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
proof_r: Proof 'r' component,
proof_s: Proof 's' component,
```

**bandersnatch_sha-512_ell2_thin - vector-1**

```
02bc4e98404607ce56429007e6ec20a2c0cb2070fd77bb377d25c83afff4670c,
05727c7bdb1a03a985bac26b26f4840730336e9f1aedcdc829e0e95ae5e80936,
-,
-,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
45ee3e316c29dc0d5c2490ad14f4c1ebafbedf897d3663406e08acaef865a586,
b963d3ab9dd2e29a172aee407cb18def053e08d43d6b50bf04544c448a3edcd6,
28ba4a40d25d6875c1e1ddb888a609156856a684513580a559496e21c9ea3cdf,
41ac847238e6b2fff7b2b476ebbb6cd6f92b024bdfa5fe8aac85f1d4c64fe913,
```

**bandersnatch_sha-512_ell2_thin - vector-2**

```
c5410a37c4dac5e7a0f02c2df9073a70b8c16d9c786c054b70f7502741c64f14,
dc9f6c647b3fe0b3248d6afb2d305d75b421b9e697e0eaff913e59dae68a76cc,
0a,
-,
a208afb3b276ef91530ce906abe1e64917b612b6e062a0bb93090c77d9c5ba95,
331bc541947a27c518a7101db25b20878bc075ccdbf15f22243d4b7cf87cd8bb,
24884e2a38ba6a7bdd6016be94cf09bc26306c0d649c67d4ed5e3ecd07ed690d,
4bbffe40e38b580372cc5dc1bfed6da3f0cbf2ff8a8058a2e0b840cffae18947,
459d5f7739b629ae8717ded2a072ec12573c0224b1197a4cb09286cb3977e51a,
```

**bandersnatch_sha-512_ell2_thin - vector-3**

```
783510ef50915b52075650aa994085d36d513e259115fa408ac67f1398814605,
689ccd15496386db306024633493e60135a2673faad2b0b4140a4006537c71b8,
-,
0b8c,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
28c86b6880842180b972a94696703d6b489f8f03a3c280d9d3047017380e0baa,
1c8c9baf9d6b691be3db3302bd64c56880f9468bb4e98ffcb545a56876717fd3,
480a2cc4b53dafd3c0f65d70c9c5dac6edb7013a81ea6516987ac38743942cf3,
0db4d5d1f6d98ec4deaf4768d5c457c57942210f422e7f3c4af167cf9e9e3a03,
```

**bandersnatch_sha-512_ell2_thin - vector-4**

```
ac162d824faac7fb45ffd6f52acc819537139a5a3027e16d2736b84e3904301c,
566dd2374ae7621fb6910143f1f5fa91b7bfce2db27a731cf110a27309b94667,
73616d706c65,
-,
f534e99a16886cb60e3672a9bcd65b57ec8a76a3d3f005850e9b38ea17d81636,
b7d8e88cdb0dfe99bba76e5693d4d562c7fa8cd06d84502582aea9f8ab0b0d8a,
8f006754011b549653e3796fda62c5fd9d0d3c5913e1737fdb638e9bf53879d2,
41b292967fe1bf7ba1edc8d33f65daf7013f78b5aaa4767ec26de9d1b969998a,
82bbd8b4649d092c2d5ecd2363e68fb80547ba6db8f7d4f23872c0fe2f9be90f,
```

**bandersnatch_sha-512_ell2_thin - vector-5**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
-,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
faca93b4f00ef37596bff4f7f23d33621cf6d75df7b4dc91615cdafb7eb8a843,
10e5943fcd69f3f470e3de7ea2edea41ce192255709048d6a3178ea17556c61b,
```

**bandersnatch_sha-512_ell2_thin - vector-6**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
c283cb32a074b3b5852dd7ed3ab23615702795a566c7eb38298eb411fce81b17,
349df96ce2eb8f99d09c26ebcf872e82bbebb77d3f1fb75d2b02accb195ba203,
```

**bandersnatch_sha-512_ell2_thin - vector-7**

```
76f8b81d866b4c8b89d1f7d40954c406ad3b6c33e2bb8ece9102d7a4f8483502,
0384ac15569e3147105862c293863fa47aed4718b5fd791cc689fbfa7e8ce75c,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
a2332c9646371ec14e3ba6ac3ae8ea93a65159dec7dcd2e5c022e14bdf7395b3,
26ea4ea18ba21e48b1c0a658fc1a4600f21c6957f6cc85e1209e69b77dc1eab6,
5a1319f8558ef88c632439acc5420f6388378621b9a10bac5206f8e6283a46bf,
9d2e74084ad6cf7b150b24955a4b7241be7d255221e366fae478556f16ec0419,
```

## C.3. Pedersen VRF Test Vectors

Schema:

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
blinding: Blinding factor,
proof_pk_com (Y^-): Public key commitment,
proof_r: Proof 'R' component,
proof_ok: Proof 'O_k' component,
proof_s: Proof 's' component,
proof_sb: Proof 's_b' component
```

**bandersnatch_sha-512_ell2_pedersen - vector-1**

```
02bc4e98404607ce56429007e6ec20a2c0cb2070fd77bb377d25c83afff4670c,
05727c7bdb1a03a985bac26b26f4840730336e9f1aedcdc829e0e95ae5e80936,
-,
-,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
45ee3e316c29dc0d5c2490ad14f4c1ebafbedf897d3663406e08acaef865a586,
b963d3ab9dd2e29a172aee407cb18def053e08d43d6b50bf04544c448a3edcd6,
3baa48811921cfb4b2bc2e4497128c3c22b13610b2ed527a075accab12834c18,
bba78d0da7a17192a42a1dbfee6053c680132082201e30d616a203a58c9631e5,
aaf19cdd5b9ae62dc329da51b06a248d3aaf38fb9b0230d663f60e5055015aef,
41efcf83c7f272a517a57b378c9b01bf50ba58ec543411c4a51575a0127fa343,
3db2b35a14d42898b779a651e6278c0f4b46172dee6cf1460e659dcd3ab7ea0e,
09b2535caa7cacfee6c75a56728e8ecb745c09f4a1bbbd1036aeee11548e1213,
```

**bandersnatch_sha-512_ell2_pedersen - vector-2**

```
c5410a37c4dac5e7a0f02c2df9073a70b8c16d9c786c054b70f7502741c64f14,
dc9f6c647b3fe0b3248d6afb2d305d75b421b9e697e0eaff913e59dae68a76cc,
0a,
-,
a208afb3b276ef91530ce906abe1e64917b612b6e062a0bb93090c77d9c5ba95,
331bc541947a27c518a7101db25b20878bc075ccdbf15f22243d4b7cf87cd8bb,
24884e2a38ba6a7bdd6016be94cf09bc26306c0d649c67d4ed5e3ecd07ed690d,
6e9876a539fbee1660541b328dbc76c421f2bb748d98f0e95aea4a38714c6b07,
f09d813c81fc2785e4ef2c5fa56a330a9985bc6b37d1056e2e2a25ed9e0faa17,
eefa655d3387ddad3df074aa3dce1155005271690192744883 08a9014b5a87e0,
1a5c65ce69f362aa542321257c31f867b1cce2baae4c85ccd8b6c996ca5b3f07,
4774cb236f639ec88560f9ccb66ec9ed08c8198dc6666abbfd80e83d7814e401,
4fc0967d0fca2b73cafbb7bb3f66714d3dbaf463541966edf3a51f69d0b7961c,
```

**bandersnatch_sha-512_ell2_pedersen - vector-3**

```
783510ef50915b52075650aa994085d36d513e259115fa408ac67f1398814605,
689ccd15496386db306024633493e60135a2673faad2b0b4140a4006537c71b8,
-,
0b8c,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
28c86b6880842180b972a94696703d6b489f8f03a3c280d9d3047017380e0baa,
1c8c9baf9d6b691be3db3302bd64c56880f9468bb4e98ffcb545a56876717fd3,
9ac473260dea264e267010a99f9e7d21f7bd135015dd52568b6be2ee35da740d,
45527e7dcab434dd93fce3fb996768a474210b9c529477c0b0b905272f813256,
4acdf452a658bda69ab725eb38271ed0816853975aa7ed5b00e40996a61605ac,
100001b838b68535c6dfeba23bec1ed0885dce43a8c424bb8be0e3c68cb1038e,
d2b8cc54885d8c8e44c966eaca8b9ae9613823e3b9a3d1088573c504770c280c,
c1f64ffcc8dcfc014d0982cd387ab730aa7c28a9c5e90e6b614b5de79e7fba0e,
```

**bandersnatch_sha-512_ell2_pedersen - vector-4**

```
ac162d824faac7fb45ffd6f52acc819537139a5a3027e16d2736b84e3904301c,
566dd2374ae7621fb6910143f1f5fa91b7bfce2db27a731cf110a27309b94667,
73616d706c65,
-,
f534e99a16886cb60e3672a9bcd65b57ec8a76a3d3f005850e9b38ea17d81636,
b7d8e88cdb0dfe99bba76e5693d4d562c7fa8cd06d84502582aea9f8ab0b0d8a,
8f006754011b549653e3796fda62c5fd9d0d3c5913e1737fdb638e9bf53879d2,
0d78247c984a435203f87c313dcc3cb30f98fe19ea1707a3f6c62213dd940314,
d51c77c0b6d7777800a693417b07cff5ecf6107640a44f872266b9b7d646abae,
7142c02e992a9f102529a981b07e033610147e17465849dd8f8a9db79b197e11,
8e1edff94266aebe0de4abc4fcd8674b1e72dde659a80f633a541b6b6d5570bd,
07886250014c2cac5bccd54d055e81b13c3bf166f2c0300a795ee0d7180c8d03,
a319912ca8e7245e3130b6e22b87444e9d2c43e2baa84ea34dc253b6b324e213,
```

**bandersnatch_sha-512_ell2_pedersen - vector-5**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
-,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
26922b938c71b84b655164000ce7fd03502101b127b588ce3e19e31ef8e98617,
3ce1329adf9ed9d724a4b7d740b0e35148dc18fb556fba83f2d86eca6527f271,
54794a95795e03575df38a2f799feade6a62bdd70abc743b976572cc0de1212e,
8b9c1b24828e8dc990bb9578d18b2d88112019600d840d3ff536078ebcf64e99,
cd541816013d05b8547e64f8ef9d0b0f41d506856e2fb89cc4bf099a31fd6003,
bedd323e94fbddf14c55a2eb04c4e686c5fa32d06797065dee00720679c7be16,
```

**bandersnatch_sha-512_ell2_pedersen - vector-6**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
e92c4361fa428aee820ffacd98b720902febe97edba1cfd865c96be4b30d6c0b,
5e90ec844e6c487d42ea66f46d03e998d9352a3f18fa01d13f8907504070f389,
f6cdc963a17fd2807b3bd3e638561c8a4c276d7009c227ddc56be12e997e0285,
867275332775f5c505668e5503aadaa29ee4ba6aed58a8c63988f308bdbfa910,
6d20d2c4ff7347c8fcd7628a94629374664ef99ef5ce185d6351d9b015cef502,
010f092028ea81e258c2517c3a685fa7e4eb56404831fa08450d4d1e525d680b,
```

**bandersnatch_sha-512_ell2_pedersen - vector-7**

```
76f8b81d866b4c8b89d1f7d40954c406ad3b6c33e2bb8ece9102d7a4f8483502,
0384ac15569e3147105862c293863fa47aed4718b5fd791cc689fbfa7e8ce75c,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
a2332c9646371ec14e3ba6ac3ae8ea93a65159dec7dcd2e5c022e14bdf7395b3,
26ea4ea18ba21e48b1c0a658fc1a4600f21c6957f6cc85e1209e69b77dc1eab6,
f67e235556416f9c25dfd7cb5a20e604b61a33873fd45a5e49859c637097e515,
2f66b036b6b94aeb23ca08f973de9975be8e3ef70c04a02ee7b774ec0aacf68b,
f5a2c3b55fdbde96f666bfa678c1e61c3c286d0ea340753ba47e345c1bef42a4,
4d763f0e68dad33df0793d563927e3be54bbc5a3a14c1aebe53b5376d1420e12,
17e28d896b23033642617a170c17f442ec5ee1a38d7957970b352fe7837e8f0b,
c74fa25391398ed8aed9260de6774d8a84fa7e496be2fa71cc07c2521129d60f,
```

## C.4. Ring VRF Test Vectors

KZG SRS parameters are derived from Zcash BLS12-381 powers of tau ceremony.

The evaluations for the ZK domain items, specifically the evaluations of the last three items in the evaluation domain $\mathbb{D}$, are set to 0 rather than being randomly generated.

Schema:

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
blinding: Blinding factor,
proof_pk_com (Y^-): Pedersen proof public key commitment,
proof_r: Pedersen proof 'R' component,
proof_ok: Pedersen proof 'O_k' component,
proof_s: Pedersen proof 's' component,
proof_sb: Pedersen proof 's_b' component,
ring_pks: Ring public keys,
ring_pks_com: Ring public keys commitment,
ring_proof: Ring proof
```

### bandersnatch_sha-512_ell2_ring - vector-1

```
02bc4e98404607ce56429007e6ec20a2c0cb2070fd77bb377d25c83afff4670c,
05727c7bdb1a03a985bac26b26f4840730336e9f1aedcdc829e0e95ae5e80936,
-,
-,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
45ee3e316c29dc0d5c2490ad14f4c1ebafbedf897d3663406e08acaef865a586,
b963d3ab9dd2e29a172aee407cb18def053e08d43d6b50bf04544c448a3edcd6,
3baa48811921cfb4b2bc2e4497128c3c22b13610b2ed527a075accab12834c18,
bba78d0da7a17192a42a1dbfee6053c680132082201e30d616a203a58c9631e5,
aaf19cdd5b9ae62dc329da51b06a248d3aaf38fb9b0230d663f60e5055015aef,
41efcf83c7f272a517a57b378c9b01bf50ba58ec543411c4a51575a0127fa343,
3db2b35a14d42898b779a651e6278c0f4b46172dee6cf1460e659dcd3ab7ea0e,
09b2535caa7cacfee6c75a56728e8ecb745c09f4a1bbbd1036aeee11548e1213,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..05727c7bdb1a03a985bac26b26f4840730336e9f1aedcdc829e0e95ae5e80936
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
```

```
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
b401855f407196cec76b6c590dc71bea696e51cbc5ccb8e6f1b1401ebcaf183d
..76fa505cc4345333694f2107d450e94fac4f3451c8d4e88c406dbc452f4ff4d4
..7b7391b57008e7f29a36cf7deaddd4307a03380b5130334d95ac1f87d7b2b8f1
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
95b258ec27b2ed9a87c301b79da6560a940f7d6d45930473db28c916b3e88413
..565d40fbd827f1dbef670d12f18c86519107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..870f9d86ed2d5c46ef71043f844d0a961b222785d2ffa76f6e4c74837c14fd57
..728f70ec0d732ec04c3e0ae7416f2f0a8ebbabf471c8fce178f72a8128addb59
..a4ca61e6cf331da22af57ce69c6da646274b02c20ad451983080798b97adf488
..e8584a18f1b121feaa9ffebfb75aa10fc2742b9144d6b374d228b01888ec9715
..3849fefc97b02f473bc56327b39abe990c623980e901e5646542c6c00d013215
..379e965632b83010e0c01c779df60dc3f6161a73df92754a31393a466c980b53
..ab8f0e80fdab4f91055762afcce5405c5592594838a959df7e7bdf601aedbe15
..b153ab818fb80646834de7157988fe240544dd2bbe7d5051f8b047500aaf2706
..4f09adab4445dd10ad3a9fa2ae619ace5f4d41b4d8f3b37892d280ff5182b447
..bc1e33a42d2ac7affe8bfb3cfd8e09bed824832f23cc44b840141312bc8bee58
..88b9c680dd5695c592b8ac3ad93e518ec17811e80191c8a63d5d6c506508d77b
..7aee157a9e93ff693d19912f470e5e05dba2be49a180eadbb7071dc3846777dc
..d2ff1bb44b7e48e65d1893cfa34aa16c82ea85c305f7941711a2a466946e13b5
..05c472bd1b8299e2f70d864a5c76000e1f9f5bd36b622f0f4fb4dc7bab87b7c3
..95db5288f2db3e004600ec1d4252ea477fccdfb67dd5eee658af784de3e4ebea
..138b33e46287cab03b7886c7b836963f,
```

**bandersnatch_sha-512_ell2_ring - vector-2**

```
c5410a37c4dac5e7a0f02c2df9073a70b8c16d9c786c054b70f7502741c64f14,
dc9f6c647b3fe0b3248d6afb2d305d75b421b9e697e0eaff913e59dae68a76cc,
0a,
-,
a208afb3b276ef91530ce906abe1e64917b612b6e062a0bb93090c77d9c5ba95,
331bc541947a27c518a7101db25b20878bc075ccdbf15f22243d4b7cf87cd8bb,
24884e2a38ba6a7bdd6016be94cf09bc26306c0d649c67d4ed5e3ecd07ed690d,
6e9876a539fbee1660541b328dbc76c421f2bb748d98f0e95aea4a38714c6b07,
f09d813c81fc2785e4ef2c5fa56a330a9985bc6b37d1056e2e2a25ed9e0faa17,
eefa655d3387ddad3df074aa3dce115500527169019274488308a9014b5a87e0,
1a5c65ce69f362aa542321257c31f867b1cce2baae4c85ccd8b6c996ca5b3f07,
4774cb236f639ec88560f9ccb66ec9ed08c8198dc6666abbfd80e83d7814e401,
4fc0967d0fca2b73cafbb7bb3f66714d3dbaf463541966edf3a51f69d0b7961c,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..dc9f6c647b3fe0b3248d6afb2d305d75b421b9e697e0eaff913e59dae68a76cc
```

```
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
a9737168dba5216251d0d8441bb678fad4e07f2f1dc49cdcdf2c68132326ec95
..d4818d3a1070717d86f8fa24c3bb0e73a4ebb67f21ce81738e570727795c4553
..01653e09c298ee4807d32a4dd4622da1caaed3086894a449b8b179466a2f78e7
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
8407fcfc2c67217d296afcc7250c43d022679916c6ef97c3fc7c794a2708ac83
..4cdbcead0ee3a5e778250ea0aa28bb679107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..8658cbda1a56b79bd5da30be2fb5d9af07972a56a2a7a141578d5ac72514061a
..2781c83f24d9064d3384923fdbff78709464d871fefb06acfddbf2b8f15ed342
..6fd3dd0f8adc3ad55bf9fdfbf8f9e3b8d78394a384733291df08a858e6cdddf2
..066d561c7ae311b9b6f5ffa659afac7df16a88c7e955bc4662e9deec298ac703
..58375652c6231760cbbd0f973cfd34d59aca627855b58945a662d74ce4fd0b55
..0cec19e41a2eb6a53723a41e662c8ee6acff299b29bf297fadc2e3ce067a6617
..f24de897276119a649ed740c02f1bf0b7746b13fe1dbfcd429a805b8e9199a69
..c1e7b18570a0ce1a087d2dd91f4346da6f92a758ba429db8ee8fd6006e466372
..e191627c6007950d91cdca0253d8c39361342176fc66495b713bebfad8fff11b
..8456304fb5c44feba9318307edbf0c9a81be8bccf8a7f7b8aa5ae7bf420eb648
..b41e31be788d3eee93654f76391ef566b8d906fc68f9b659a835658896a7dec4
..2514ff52420505d3c155d945d4a01ffcb6eb3c081768863ca2e33ad3ff858a27
..16edd6f497c2798e51729605c3d7701498924025465b94153552c3dfdfe11f69
..e03efdf62e54d367f4b086fdeb70411c729a53ed3436f29b9924c8422d81b10e
..99538490c3cdd1b150d6dae0c9c647dc0b2ed69a30134779b89114e1605d622d
..aae5d8b8fba690997e9bc7e369ec05fa,
```

**bandersnatch_sha-512_ell2_ring - vector-3**

```
783510ef50915b52075650aa994085d36d513e259115fa408ac67f1398814605,
689ccd15496386db306024633493e60135a2673faad2b0b4140a4006537c71b8,
-,
0b8c,
9646c575af23c811c5d4691a26121c845b80a3805452c7267c2dbe2110baa6e3,
28c86b6880842180b972a94696703d6b489f8f03a3c280d9d3047017380e0baa,
1c8c9baf9d6b691be3db3302bd64c56880f9468bb4e98ffcb545a56876717fd3,
9ac473260dea264e267010a99f9e7d21f7bd135015dd52568b6be2ee35da740d,
45527e7dcab434dd93fce3fb996768a474210b9c529477c0b0b905272f813256,
4acdf452a658bda69ab725eb38271ed0816853975aa7ed5b00e40996a61605ac,
100001b838b68535c6dfeba23bec1ed0885dce43a8c424bb8be0e3c68cb1038e,
d2b8cc54885d8c8e44c966eaca8b9ae9613823e3b9a3d1088573c504770c280c,
c1f64ffcc8dcfc014d0982cd387ab730aa7c28a9c5e90e6b614b5de79e7fba0e,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
```

```
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..689ccd15496386db306024633493e60135a2673faad2b0b4140a4006537c71b8
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
b587fc8452db2dab040f7e0ce6c89be2b39d3e584460a7427367d5d71f030b3b
..c42cd74b6d19544b9d6ac677574cbd9cb64007d347b9f90a19e94f3e2402ce21
..6cec8f85822d2c275d4aa97fdd6609463752fc36886c6afa91369a375c7443e7
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
920e0198311d84368a35db10aa11a9b3b4b1b46576e2301053b9eba4a79ef581
..43726ffcef2c40cbe23d731aef19405b9107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..8ea1aca0088bf3b2cc56a1f9bb16c7f282d7fe7ffd96fe6dbb8b3f16cd9392bb
..fa44cf48987c43d042d99bdf5e0e7214829eb8da35ddde985d6e1a081ff16112
..e8f6ee0c2cfc9e90fc848d1cc7851004e5734ddf4ac16eb0f3650fc096e673f5
..2ccd1c9dc58a4ee502a587b5cb5d9bcd9c2f0ce46a8c974a153048c342593772
..c62c17fc562c7dd879605fbe02bb37005130b0b4f09fdd9e4a3cdc9118296e35
..753ccbf75d9b319db28e6db54186f66d09a2cd53a0a69be774bb9aca73bacb5c
..8bdedd4de42f9f94168ed3a1cdb29559bc17ff1810ff52bce6b25d05dab1e122
..35d92acf8abb5247e828ab04588d975ff6e9baf9633751e17c295c378c11d003
..b790b787ae919d1669c094e0ae917b4d8f5a2f1e711d96bdaa8db7f2b8276d36
..b1a57266168d6e6a3bb11f2823689ebed001fa0dffb991b7247922e31a9e4006
..b96d9369260bcba44f683441ab1142d0bdff47b56ff1550b5672f79dd576661b
..ec2af6969760312e4b0ad73e00e1f1e2b5ca41a15eed5766384a56e7e58edf69
..f462b1b9a6a7f7725bc59fe36351d64da33dc4567e5e83043c4211028abe6cb7
..25ed0fe845cdec5a7018b975a3d675b3543fc04a7907359487a907a751be5bec
..8e8433a3bc0d66d9217b1c2e4501f4ff1b6ed2954f99584de6bff104b957df1e
..c122422adfce5e19f27253385f410ce3,
```

### bandersnatch_sha-512_ell2_ring - vector-4

```
ac162d824faac7fb45ffd6f52acc819537139a5a3027e16d2736b84e3904301c,
566dd2374ae7621fb6910143f1f5fa91b7bfce2db27a731cf110a27309b94667,
73616d706c65,
-,
f534e99a16886cb60e3672a9bcd65b57ec8a76a3d3f005850e9b38ea17d81636,
b7d8e88cdb0dfe99bba76e5693d4d562c7fa8cd06d84502582aea9f8ab0b0d8a,
8f006754011b549653e3796fda62c5fd9d0d3c5913e1737fdb638e9bf53879d2,
0d78247c984a435203f87c313dcc3cb30f98fe19ea1707a3f6c62213dd940314,
d51c77c0b6d7777800a693417b07cff5ecf6107640a44f872266b9b7d646abae,
7142c02e992a9f102529a981b07e033610147e17465849dd8f8a9db79b197e11,
8e1edff94266aebe0de4abc4fcd8674b1e72dde659a80f633a541b6b6d5570bd,
07886250014c2cac5bccd54d055e81b13c3bf166f2c0300a795ee0d7180c8d03,
a319912ca8e7245e3130b6e22b87444e9d2c43e2baa84ea34dc253b6b324e213,
```

```
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..566dd2374ae7621fb6910143f1f5fa91b7bfce2db27a731cf110a27309b94667
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
8cb872d59c5ee4e533b40fcc716092af6582109d7a0183b17b336abc537340cc
..acba3a0b124fa009fad0f1f890409be8894cd4f5d91e6b331cec686be9b86784
..90d992533388072fa898da191f7c1b8b45fd534223f243d571cdb80d5770dc8f
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
b366dc29a580115ee67751e4c85b169ea4832f55e168ec6308efd1ff25d0f613
..cd86c575aad8716d96f5521f15acdbf39107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..97ffe13e501d54b33dcbbcb13862729586f430dd03c382aa91742d0e1f81b62d
..30e5be3379ea488ce970510713fcca08b106ea47a709fe3d055cce53d1fece85
..6709c0c2f1e34ff845e4bb9056db0526be9531134b37563aba7f4d8cc178c737
..840d1bef277c946abbb2e6cdf1f17980a86f3baea16249e60e75247a2d140964
..a4382e2006070440f8abcd052372a1a5bd17b3b63bb5419965af5589aaa34908
..5b10f5308b51aa7927f048f3f8f0dd35e28e22ce3d426b68ca33b0ce0ebbc761
..fe9c3ced5867e579abd8cd62125f31420f70b3a7bfcbbcf57e405fe0f491b621
..74805c7be10b79c33cfdeb9778ef1b20fea34d7a80d179694d1bd61c1e351009
..2fa950461560e73c138718955eeea49c41ef7c98479068443330337e90664d61
..5d2b87d63063e3aab73d60bd803528e6b9fc5c8b9997d2326dbc86742826a344
..84c781f1fb236b2ba258b305c63794fd6eb4e4ec3db4a31e67fb0e3aee89d4a0
..858c8159381c2cad1ef56d018a7d72dbc85a3e049e01fda527bb533192e7a64a
..bb065d195e2caaf285a12f80c20d264cafaa0ecd22a57a0403d112247daf9cca
..47dbdb971c72788f561cc83976da04f1e3f66544bf47afa1eab80b2e6b33b71c
..91f99b01705be9f0408274066fab8b8aee773577a316f76858ae59ca75b0db0e
..2a90064290ecffd315ba64fb28b9e067,
```

**bandersnatch_sha-512_ell2_ring - vector-5**

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
-,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
26922b938c71b84b655164000ce7fd03502101b127b588ce3e19e31ef8e98617,
3ce1329adf9ed9d724a4b7d740b0e35148dc18fb556fba83f2d86eca6527f271,
54794a95795e03575df38a2f799feade6a62bdd70abc743b976572cc0de1212e,
8b9c1b24828e8dc990bb9578d18b2d88112019600d840d3ff536078ebcf64e99,
```

```
cd541816013d05b8547e64f8ef9d0b0f41d506856e2fb89cc4bf099a31fd6003,
bedd323e94fbddf14c55a2eb04c4e686c5fa32d06797065dee00720679c7be16,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
89bd64badc4998bd91dc2eae3edd5e90e60ae6d3ee9722d43822c9653eb1c0dc
..19d24533e761b39fa68e6ab92a3ccb79b7f681bde365bd3827eacc69dd8ed46e
..fc530532b2110eb097360a6db718dd233580a7880a560d794dbf63c5fc146c6a
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
a42ac27fbd3e2a57863603b03ae5cdf313304212981574c50fa8e07082bfeaaa
..1eb24f6b5cbe4b394bac56bf847558a79107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..8173c4b1dc29fcb6f996510f5bf6149c9a17e84bf10a48fe74aaa3d12907066d
..77aa7e9b1aed7ae8b779f002189a4490b3f302ea4518fbe9a45c887d6985773a
..b61acdae9a2192cdf14cb274f876fc262ff3439e7b3f011bc33a8058c6f193aa
..93277e6061f6938ef9ed9b10aebe4a3e47897b489d687fc42525beb4e8c2d21b
..3722a40dee65c20cd4da0c56267307ecfd1d34c34570ef3cca6180a41a32b810
..4523ecbe2a277741e08e9c5a12282cbd1f80d1c72f50c332bf86d4cb83905a64
..74b9a75ebfa4b836da18140ae779e75d4f530b7a4cc1b87e3517b1fd8a8ce969
..7fdda8ea884eabfe0c7a9d7c229954f8b1948fc158cd7a2518db4e55d21c7a2d
..55747650e4b1d98b16a9da6290f7913da415c71f7a3cdc3e1c83d57d7cf6d867
..73d14e95272013a41d5e3bbf7551aa6fb9c56c45edda82ed4d07ae2b03ddfa2c
..b051abc16e8efd62d569ee0cdef88a61067e2a1297de6b22a40bd782be1e9a33
..8424362061c0c25d47cdc8c096046e0031b040af06031ec3496352555f1e0e3c
..f6699ab72d136fec837c4684dcb51d5c92a0cbc94496ae8044fbb409cca0a8ed
..e879ca16a81db941c376a13f9a6776b803f3e277a1fdbf8d133b0b4b88649bc3
..897aa7d664109a5fce8517465a06eb9ca370a0c395def125abf3b2dd62997b4a
..cd33e6dc8c16a4bbe6dc2f45d34386a9,
```

### bandersnatch_sha-512_ell2_ring - vector-6

```
9b44b39c5b801e2c4c8d6ffec1438896d0dc85aa7105e09ddde213dca8112515,
0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
819314355518debccaa1c713a71968575353cdd52d03a9fc8eca4bb2b21aa562,
2b544ee79c45df398ea8c6abb24bc2915ff7055b20962f1cd26a783408d5f8f5,
e92c4361fa428aee820ffacd98b720902febe97edba1cfd865c96be4b30d6c0b,
5e90ec844e6c487d42ea66f46d03e998d9352a3f18fa01d13f8907504070f389,
```

```
f6cdc963a17fd2807b3bd3e638561c8a4c276d7009c227ddc56be12e997e0285,
867275332775f5c505668e5503aadaa29ee4ba6aed58a8c63988f308bdbfa910,
6d20d2c4ff7347c8fcd7628a94629374664ef99ef5ce185d6351d9b015cef502,
010f092028ea81e258c2517c3a685fa7e4eb56404831fa08450d4d1e525d680b,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..0505219b2b79f731c6ab9cda8e1c0d02f7f8862a6b7c7cdeb598f69e24edc9f1
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
89bd64badc4998bd91dc2eae3edd5e90e60ae6d3ee9722d43822c9653eb1c0dc
..19d24533e761b39fa68e6ab92a3ccb79b7f681bde365bd3827eacc69dd8ed46e
..fc530532b2110eb097360a6db718dd233580a7880a560d794dbf63c5fc146c6a
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
8365796b6a7889f0a7c96b502d0b26282c8531a1910defdcb3181e272e1c3b38
..3494b8e274060ca2287f164d3dd34ba39107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..b3d4ed85e4a7868b15d1f9dbdda94349e5ec4b15e2f7ebf92db5a5ad5c5ac9a4
..b4f6e7f5d46939d35e39dce2818b55d98a0cdf62a1b922ba440480f9bba6f719
..b3dd89798ae6bcbf927ed6001c6792e6882e13f0b9a793986131c657ea2d9668
..169046f22d0bf9cd755e1513d09b4426b2841f1db5fc7ff18e57be60c4f4d053
..e1ad4cbb9f01b18ca23cfce84dcebb070c4520b5445d209aa1835e108c306429
..f0a2a0d0a963d35dd30a9757d9c8f71ca8194a36921df7bae7f9669d91df1d37
..61dbeb4d77674e4151c49573cebb4b084ac3ad453bcd00df15f9dc02d2b13461
..07fc58774d8856608ed661abd380aeeaa0254ba6a42f6f6b521c0d65c1e25245
..3da83a02298e3739d136df2ff9b7bd647dcfd631213620e4b098f2564c2fe20c
..520b3d3b5000b71486d9dd2421a297436dd5a91658f8120ea9e7c47709fdbe71
..969430c473503030e6a4618bd443f0f2dc81798e6de9660b6f0cb4ab816e53da
..70f4c26f671bff6f3e1e03e4764a4977ba67d4a89a4c3eb868ff874eadcc0f9f
..368812228f6a9e6ed04cec7bbd356401b597b5f7ba7a5f3a9b415d4be6690b3d
..c8c5e3671c434830dde1bd672211ea3e146aa04ddd1317f253821dd780c554ab
..92bc8ac170b8050e3a7b2a3c8d835ab13ac70012ae382c09353b359e99d246ad
..e99de5bfc79e10e715c5b24723990a4e,
```

**bandersnatch_sha-512_ell2_ring - vector-7**

```
76f8b81d866b4c8b89d1f7d40954c406ad3b6c33e2bb8ece9102d7a4f8483502,
0384ac15569e3147105862c293863fa47aed4718b5fd791cc689fbfa7e8ce75c,
42616e646572736e6174636820766563746f72,
1f42,
47a25c71b9fccb60c72c2f5f9851df8594a9d346cda259dcef6ab5e6e441b795,
a2332c9646371ec14e3ba6ac3ae8ea93a65159dec7dcd2e5c022e14bdf7395b3,
26ea4ea18ba21e48b1c0a658fc1a4600f21c6957f6cc85e1209e69b77dc1eab6,
```

f67e235556416f9c25dfd7cb5a20e604b61a33873fd45a5e49859c637097e515,
2f66b036b6b94aeb23ca08f973de9975be8e3ef70c04a02ee7b774ec0aacf68b,
f5a2c3b55fdbde96f666bfa678c1e61c3c286d0ea340753ba47e345c1bef42a4,
4d763f0e68dad33df0793d563927e3be54bbc5a3a14c1aebe53b5376d1420e12,
17e28d896b23033642617a170c17f442ec5ee1a38d7957970b352fe7837e8f0b,
c74fa25391398ed8aed9260de6774d8a84fa7e496be2fa71cc07c2521129d60f,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..0384ac15569e3147105862c293863fa47aed4718b5fd791cc689fbfa7e8ce75c
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
b3cfb7d73f916c3e9b19020ced68c34f3fd354fa57b73e0f47fcd3a0b37fc452
..7d3db88335c1b7dcdef0be19176bfd8a93f93e869d1ec517a9470334f65bfd1c
..2886c6e1f73be76cea825f94a13201b59081999eb97cdb939ee134f117bfd142
..92e630ae2b14e758ab0960e372172203f4c9a41777dadd529971d7ab9d23ab29
..fe0e9c85ec450505dde7f5ac038274cf,
ab5f081934cd9109aa080532529085929b2621db492017375ff51f415565b88b
..403a3c628916055ba9359877747ddad59107bd20fe94a01157764aab5f300d7e
..2fcba2178cb80851890a656d89550d0bebf60cca8c23575011d2f37cdc06dcdd
..89e3a57b4f2d76b387c802905086cb3fb4207c69acc9a004f6329b9635ee076b
..532c39cf8f4860aa2065b2a9a7a421a0b6620c2f8cd96ea1deaf3a815b73920d
..33d66bff55efeaec6a675ce8d0a791b4e0965d6e874d58a111001bf8827c70f0
..974d2c9e7ea44a66c4f8a0cd1435cafa61dbc4db429146561e4271b23d041d32
..09801f8b9e10b26e47904a549989bfd3002fac5a280ae6b2d602afa72e1ded0f
..36eaa24e23b5ecbf637579580cb0664500d357e2c5ac9e523a7867720bf03d3c
..e2019e0529b67592c72656fe4d4f01f7d2aea8545248354fddeefa12b4cf7d0f
..280b32208c44c8cf62a0a994ca35c835330836a008fef26cd03652aa13e78828
..f6a1230672869e39602cde2af655ad9623a12ba300d832ef8265e7949e3b5b07
..7a43b59f26f3c9fc006930af7b0e6e76ea81a20673d9def754ffeb2086ee6528
..b8ff0af3e0fe9453d10b26a93403ddaee0d83c15d0b95637b006955c3cccbd4f
..128c6c5eb62b20c177ea424e6e47d1f40496ffc8af31f2d6421126583f903865
..8ebaba6a1c31dc5d70e489aaa22a344da08fa7f3cd61646994fcf49622f053c1
..09625f8585163124a2b736b51ce6c00ba7383ea69b6969328235ce0aeaf66bad
..8714847026c23163de5b8b506e0474a606a0e9429359cacb54aeefd2305fbce8
..eacf07e33d5e1032b7de4e6f854ba1d1,

# References

1. Internet Engineering Task Force *Verifiable Random Functions*; RFC Editor, 2023; https://datatracker.ietf.org/doc/rfc9381.

2. Burdges, J.; Ciobotaru, O.; Alper, H.K.; Stewart, A.; Vasilyev, S. Ring

Verifiable Random Functions and Zero-Knowledge Continuations 2023; https://eprint.iacr.org/2023/002.

3. Ciobotaru, O.; Shirazi, F.; Stewart, A.; Vasilyev, S. Accountable Light Client Systems for PoS Blockchains 2022; https://eprint.iacr.org/2022/1205.

4. Masson, S.; Sanso, A.; Zhang, Z. Bandersnatch: A Fast Elliptic Curve Built over the BLS12-381 Scalar Field 2021; https://eprint.iacr.org/2021/1152.

5. Vasilyev, S.; Galassi, D. Ring Proof Technical Specification 2024; https://github.com/davxy/ring-proof-spec.

6. Internet Engineering Task Force *Hashing to Elliptic Curves*; RFC Editor, 2023; https://datatracker.ietf.org/doc/rfc9380.